

Fase 2 – Projeto 4

EasyDesk – Fluxo de Suporte Inteligente

O desafio desta etapa

Neste projeto, você vai construir uma aplicação que lida com algo muito próximo da rotina real de empresas, equipes técnicas e operações internas: **organizar chamados, acompanhar status e tornar visível o que antes estava perdido em mensagens soltas e processos improvisados.**

O **EasyDesk** representa um passo importante na sua formação porque mostra que você já ultrapassou o nível puramente visual. Agora, o foco está em controlar **estado**, estruturar **dados**, manipular listas dinamicamente e permitir que o usuário interaja com a aplicação de forma prática, sem recarregar a página.

Ter um sistema de chamados no portfólio comunica uma mensagem forte: você já entende como fluxos reais funcionam. Você sabe receber dados, processá-los, armazená-los temporariamente, filtrá-los e reorganizar a interface conforme a necessidade de quem está usando o sistema.

É aqui que começa a ponte entre uma interface simples e a lógica por trás de sistemas maiores, como **CRMs**, **ERPs** e plataformas administrativas.

Posicionamento do projeto

O EasyDesk é um projeto de nível **intermediário**, com foco em **lógica de programação e manipulação de DOM**, duração sugerida de **8 semanas** dentro do cronograma de 6 meses e desenvolvimento **individual**.

Na sua progressão, este projeto funciona como um verdadeiro divisor de águas. Depois de dominar a base visual na Fase 1, agora você passa a dar vida à tela. É aqui que entram dois conceitos essenciais para qualquer aplicação moderna:

- **estado**: o que está acontecendo na aplicação agora
- **dados**: como a informação é organizada para ser útil

Ao construir um sistema de chamados, você prepara sua base mental para lidar, mais adiante, com bancos de dados e arquiteturas mais complexas. Mas, por enquanto, o campo de batalha continua sendo o navegador do usuário.

O cenário que justifica a solução

Imagine uma pequena empresa de manutenção ou a administração de um condomínio onde os pedidos de reparo chegam por todos os lados: **WhatsApp**, **e-mail**, anotações soltas e até post-its colados na tela.

Sem um sistema centralizado, a informação entra, se espalha e some. Fica difícil saber o que é prioridade, quem está responsável por quê e o que já foi resolvido. O resultado é previsível: chamados esquecidos, frustração de quem solicitou ajuda e uma operação que parece sempre estar apagando incêndio.

É nesse ponto que o EasyDesk faz sentido.

A dor central aqui é o **limbo da informação**. Quando não existe um fluxo organizado, o problema não é só técnico — é também de comunicação, confiança e rastreabilidade.

Seu papel neste projeto será criar uma ferramenta em que o registro seja padronizado, o andamento dos chamados fique visível e a consulta ao status aconteça de forma instantânea.

Quem está envolvido

O sistema atende a dois perfis principais.

O **solicitante**, ou usuário, precisa de um formulário simples, inclusive no celular, para descrever o problema e confirmar que o chamado foi realmente registrado.

O **técnico ou administrador** precisa de uma visão geral da operação, com liberdade para alterar o status do chamado — por exemplo, de **Pendente** para **Em Progresso** — e filtrar rapidamente o que ainda falta resolver.

O diferencial técnico deste projeto é exatamente esse: a tela deixa de ser fixa. Ela passa a reagir. Quando o técnico filtra por “Concluídos”, por exemplo, a lista é reconstruída dinamicamente com JavaScript, sem recarregar a página. É a interatividade em ação.

Objetivo do projeto

Ao final das 8 semanas, você deverá entregar um **sistema funcional de Help Desk**, executado inteiramente no navegador, capaz de gerenciar múltiplos chamados simultaneamente.

O foco não está apenas em estilizar componentes, mas em criar uma aplicação que **pensa, organiza, responde e reconstrói a interface** com base nas ações do usuário.

O que sua aplicação precisa entregar

A solução deve oferecer as seguintes funcionalidades obrigatórias:

- **Formulário de registro**, capturando o nome do solicitante, a descrição do problema técnico e o nível de prioridade (**Baixa**, **Média** ou **Alta**)
- **Painel de controle dinâmico**, no qual cada novo chamado aparece como card ou linha, contendo um identificador único gerado via código
- **Gestão de estados**, com botões para alterar o status entre **Pendente**, **Em Progresso** e **Concluído**
- **Mudança visual automática**, com alteração da cor do card conforme o status do chamado
- **Lógica de filtragem**, permitindo visualizar, por exemplo, apenas chamados pendentes ou apenas os de alta prioridade
- **Persistência local com LocalStorage**, garantindo que os chamados não desapareçam ao fechar ou atualizar o navegador

Foco técnico do projeto

Ao construir o EasyDesk, você deverá demonstrar domínio sobre:

Manipulação do DOM

Criar, remover e atualizar elementos HTML dinamicamente, sem necessidade de recarregar a página.

Arrays e objetos

Armazenar os chamados em uma lista de objetos e utilizar métodos como `.filter()`, `.map()` e `.sort()` para organizar e apresentar a informação.

Eventos

Capturar cliques de mouse e digitação no teclado para acionar funções lógicas e atualizar o sistema em tempo real.

Como o projeto evolui ao longo das 8 semanas

O desenvolvimento está dividido em quatro blocos quinzenais. Cada etapa foi desenhada para consolidar uma habilidade específica de JavaScript antes de avançar para filtros e persistência.

Marco 1: Captura e Registro

Semanas 1 e 2

Nesta primeira etapa, o foco é criar um formulário funcional que, ao ser preenchido, adicione um novo card ou linha à tela usando JavaScript.

Neste momento, ainda não haverá persistência.

O critério de sucesso é claro: o usuário clica no botão e o chamado aparece **instantaneamente** na lista abaixo.

Marco 2: Gestão de Ciclo de Vida

Semanas 3 e 4

Com a captura funcionando, você passa a trabalhar o comportamento do chamado ao longo do tempo.

O entregável inclui a implementação dos botões de **mudar status** e **excluir**, utilizando **objetos JavaScript** para representar cada chamado.

O critério de sucesso desta etapa é que, ao clicar em **Concluir**, o card mude de cor e o texto de status seja atualizado sem afetar o restante da tela.

Marco 3: O Cérebro da Busca

Semanas 5 e 6

Agora entra o coração do projeto: a lógica de filtragem.

O entregável será a implementação da **barra de pesquisa** e dos botões de filtro por status, como **Pendente**, **Em Progresso** e **Concluído**.

O critério de sucesso aqui é demonstrar domínio do método **.filter()**, mostrando apenas os chamados que atendem ao critério escolhido pelo usuário.

Marco 4: Memória e Refino

Semanas 7 e 8

Na etapa final, você dará memória à aplicação e ajustará a experiência de uso.

O entregável inclui integração com **LocalStorage** para salvar os chamados no navegador e ajustes de layout voltados ao uso em celular.

O checkpoint final exige que o sistema possa ser fechado e reaberto sem perder os chamados, mantendo tudo organizado e filtrável.

Conceitos fundamentais que você precisa dominar

Para que o EasyDesk funcione de forma fluida e consistente, alguns pilares precisam estar bem claros.

Manipulação avançada do DOM

Você deve saber selecionar elementos, criar novos cards de chamados dinamicamente e inseri-los na interface sem depender de um HTML estático gigante.

Objetos e arrays de dados

Cada chamado deverá ser representado por um **objeto JavaScript**, contendo informações como **ID**, **título**, **descrição**, **status** e **prioridade**, e todos esses objetos deverão ser armazenados em um **array**.

Métodos de array

Aqui está um dos grandes pontos de virada do projeto:

- **.push()** para adicionar chamados
- **.filter()** para implementar a lógica de busca e filtragem
- **.map()** para renderizar a lista atualizada na tela

Tratamento de eventos

Você precisará capturar o clique do mouse em botões como **Concluir** ou **Excluir**, além do envio do formulário, conectando essas ações às funções certas do código.

Persistência com LocalStorage e JSON

Será necessário transformar a lista de chamados em **JSON** para armazená-la no navegador e depois reconstruí-la ao carregar a página, preservando o estado da aplicação.

Lógica condicional e estados

Você também deverá criar regras que conectem o estado lógico ao estado visual. Exemplo: se o chamado estiver **Pendente**, a borda pode ser amarela; se estiver **Concluído**, a borda deve ficar verde.

Estratégia de estudo com IA

Neste projeto, a IA deve ser usada como ferramenta de raciocínio e depuração, não como atalho para receber o código pronto.

Quando encontrar um erro no console, em vez de pedir apenas “conserte isso”, o ideal é perguntar:

“Explique por que este erro está acontecendo e quais conceitos de JavaScript eu devo revisar para resolvê-lo sozinho.”

Esse tipo de uso fortalece sua autonomia.

Alguns prompts úteis para o processo são:

- “Eu preciso criar uma função que filtre chamados por status ‘Pendente’. Quais são os passos lógicos que o JavaScript precisa seguir para percorrer minha lista de objetos e retornar apenas os corretos?”

- “Qual a diferença prática entre usar um for tradicional e o método `.filter()` para organizar os chamados na tela? Qual é mais eficiente para o usuário?”
- “Explique como o `JSON.stringify` transforma meu array de chamados em algo que o `LocalStorage` consiga guardar no navegador. Por que não posso salvar o array diretamente?”

Você também pode usar a IA para simular casos de borda:

“Aqui está minha função de busca. Quais cenários, como campo vazio ou busca sem resultados, eu esqueci de tratar para que o sistema não trave na tela do celular?”

Depois que tudo estiver funcionando, vale usar a IA para refatoração:

“Como eu posso organizar melhor meu arquivo JavaScript seguindo boas práticas de Clean Code?”

O que você deve aprender ao final deste projeto

Ao concluir o EasyDesk, você deverá demonstrar de forma independente as seguintes competências:

- **pensamento algorítmico**, decompondo problemas como filtros por status em etapas lógicas menores
- **domínio de estruturas de dados**, sabendo usar objetos para representar entidades reais e arrays para organizar coleções
- **renderização dinâmica**, atualizando a interface em tempo real sem recarregar o HTML
- **gestão de fluxo de dados**, controlando como a informação entra, é validada e armazenada temporariamente
- **implementação de filtros e buscas**, aplicando métodos como `.filter()` e `.includes()`
- **persistência no navegador**, dominando `LocalStorage` e conversão de dados com JSON
- **depuração de lógica**, lendo o console e entendendo o fluxo de execução do JavaScript

Requisitos e restrições

Para garantir que o aprendizado fique centrado na lógica do navegador e na manipulação de dados com JavaScript puro, o projeto possui regras claras.

Requisitos funcionais

O sistema deve conter:

- **entrada de dados** por meio de um formulário com validação simples, que impeça chamados sem título ou descrição
- **identificação única** para cada chamado, gerada automaticamente via código, como com `Date.now()`

- **filtros em tempo real**, permitindo alternar entre **Todos**, **Pendentes**, **Em Progresso** e **Concluídos** sem recarregar a tela
- **persistência local**, garantindo que chamados reapareçam após atualizar a página ou fechar o navegador
- **feedback visual**, indicando o status de cada chamado com recursos como borda amarela para pendente e verde para concluído

Requisitos técnicos

- Toda a lógica de criação, filtragem e deleção deve ser feita em **JavaScript Vanilla**
- A manipulação da interface deve usar métodos como **createElement**, **appendChild** ou **innerHTML** com cautela
- O armazenamento deve utilizar **localStorage.setItem** e **localStorage.getItem** com conversão por **JSON.stringify** e **JSON.parse**
- Como diferencial, recomenda-se tentar aplicar **event delegation**, usando um único escutador de eventos no elemento pai da lista para gerenciar cliques em **Concluir** e **Excluir**

Restrições

Não é permitido:

- usar bancos de dados externos como **Firebase**, **Supabase** ou **MySQL**
- utilizar frameworks de UI como **React**, **Vue**, **Angular** ou **Svelte**
- utilizar frameworks CSS como **Bootstrap** ou **Tailwind**
- utilizar bibliotecas de componentes prontos, como calendários ou seletores prontos

Recursos e documentação para sustentar o desenvolvimento

O kit de referência do EasyDesk está voltado para manipulação lógica de dados e armazenamento no navegador.

Lógica de arrays

Na **MDN Web Docs**, foque especialmente em:

- **.filter()**, essencial para a busca e os filtros por status
- **.map()**, ótima forma de transformar dados em elementos visuais
- **.find()**, útil para localizar um chamado específico pelo ID

Interatividade e DOM

- **JavaScript.info**, especialmente o capítulo sobre DOM, para entender como o navegador organiza a interface e como você pode interagir com ela via código
- estudos sobre **Event Delegation**, para gerenciar múltiplos botões com menos custo de memória

Persistência de dados

- documentação sobre **Window.localStorage**
- materiais sobre **JSON.stringify** e **JSON.parse**, para empacotar e recuperar sua lista de objetos

Ferramentas de diagnóstico

- **Chrome DevTools – aba Console**, para localizar erros de lógica
- **Chrome DevTools – aba Application**, para visualizar em tempo real os dados salvos no LocalStorage

Riscos e pontos de atenção

À medida que você começa a manipular dados dinamicamente, surgem bugs que não aparecem em páginas estáticas. Antecipá-los faz parte do trabalho.

O sumiço dos dados

Um erro clássico é salvar corretamente no LocalStorage, mas esquecer de carregar esses dados quando a página abre. A função de carregamento precisa estar entre as primeiras ações da aplicação.

Duplicação de cards

Se a função de renderização não limpar a lista antes de exibir novamente os itens filtrados, o usuário verá chamados repetidos. Antes de rodar **.map()** ou **forEach**, limpe o container principal.

Conflito de tipos

O LocalStorage só guarda texto. Se você tentar aplicar **.filter()** em algo que ainda está em formato string, antes do **JSON.parse**, a aplicação vai quebrar.

IDs duplicados ou inexistentes

Sem um identificador único, excluir um chamado pode afetar o item errado — ou todos de uma vez. Usar **Date.now()** ajuda a garantir unicidade a cada criação.

Falta de feedback no formulário

Se o usuário clica em enviar e nada acontece visualmente, há uma boa chance de clicar várias vezes e gerar duplicações. Após uma ação bem-sucedida, o sistema deve limpar os campos ou exibir um feedback claro.

O console é o seu guia

Ignorar mensagens em vermelho no console é uma forma rápida de travar o projeto. O navegador está indicando exatamente onde a lógica precisa ser revisada.

Critérios de aceite

O EasyDesk só será considerado concluído quando todos os itens abaixo forem validados.

Checklist de qualidade

- **Entrada validada:** o sistema impede a criação de chamados vazios ou sem as informações essenciais
- **Identificação única:** cada chamado possui um ID exclusivo, sem conflitos em ações de deleção ou edição
- **Manipulação de estado:** ao clicar em **Concluir**, o status muda imediatamente na tela e também no LocalStorage
- **Filtragem precisa:** ao selecionar **Pendentes**, apenas os chamados corretos são exibidos, sem duplicação nem lixo visual
- **Persistência real:** ao atualizar a página com **F5**, todos os chamados e seus status permanecem exatamente como foram deixados
- **Limpeza de interface:** após cadastrar um chamado, os campos do formulário são limpos automaticamente
- **Código JavaScript puro:** a aplicação funciona completamente sem bibliotecas externas
- **Responsividade funcional:** o sistema é utilizável em celular, com botões adequados ao toque
- **Console limpo:** não há erros em vermelho durante o uso normal da aplicação

Possibilidades de expansão

Depois de entregar o escopo obrigatório, você pode aproximar o EasyDesk de um software mais comercial, adicionando camadas extras de valor.

Barra de pesquisa em tempo real

Permitir que o usuário filtre os chamados conforme digita o nome do solicitante ou o título do problema.

Contador de chamados

Criar cards no topo da tela com total de chamados, quantidade de pendentes e quantidade de concluídos no dia.

Prioridade visual com cores

Além do status, destacar a urgência por meio de ícones, espessura de borda ou outro recurso visual.

Ordenação por data

Fazer com que os chamados mais recentes apareçam automaticamente no topo.

Exportação para CSV

Adicionar um botão para baixar a lista de chamados em formato compatível com Excel.

Modo edição

Permitir que o técnico edite a descrição de um chamado já criado sem precisar excluí-lo e recriá-lo.

Animações de transição

Aplicar transições em CSS para que cards desapareçam suavemente ao serem filtrados, em vez de sumirem de forma brusca.

Fechamento

O EasyDesk é o projeto em que a interface deixa de ser apenas uma tela que mostra informação e passa a funcionar como um sistema que organiza trabalho real.

Ao concluir esse desafio, você terá dado mais um passo importante: entender que desenvolver não é apenas montar componentes, mas criar lógica, controlar estado, preservar dados e transformar ações do usuário em respostas claras na interface.

É esse tipo de projeto que começa a aproximar seu portfólio da realidade de produto.